PROJECT REPORT
SOCIAL NETWORK FRIEND FINDER



UNIVERSITY INSTITUTE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SUBMITTED BY:

Varsha Vadehra | 25BCS10278
Garima Sharma| 25BCS10076

Section/Group: 802B

Semester: 3

Program: B.E CSE Core

SUBMITTED TO:

MD Tauseef Sir

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Mr. Anmol Ray (E19861), Technical Trainer, for his valuable guidance and support throughout the completion of our team project, "SOCIAL NETWORK FRIEND FINDER."

We are also thankful to Chandigarh University for providing the opportunity and resources required to complete this project as part of the subject Advance Data Structure (Subject Code: 25CSH-202), Semester 3rd, Session July–December 2026.

Finally, we would like to acknowledge everyone whose support and encouragement helped us complete this project successfully.

ABSTRACT

This project presents the design and implementation of a Social Network Friend Finder using C++ and a web-based visual interface. The primary objective is to model a small social network as a graph in which users are vertices and friendships are edges, and to let a user build and explore that network.

The system stores the network as an adjacency list implemented with an ordered map of sets. Breadth-First Search (BFS), using a queue, is applied to find the shortest chain of connections between two users and to traverse the network level by level, while Depth-First Search (DFS), using recursion, explores the network as deeply as possible along each chain of friends. Mutual friends are found by set intersection, and friend suggestions are generated from friends of friends.

.

INDEX

1. Introduction
2. Problem Statement
3. Objectives
4. Data Structure Selection and Justification
5. Algorithm and Flowchart
6. Working Implementation (C++ Source Code)
7. Sample Input and Output
8. Time Complexity Analysis
9. Project Report and Demonstration
10. Conclusion
11. References

1. INTRODUCTION

Social networks such as Facebook, Instagram and LinkedIn connect millions of people, and features like "mutual friends" and "people you may know" are built on a simple mathematical idea: a graph. In a graph, each person is a vertex (node) and each friendship is an edge connecting two vertices. Because friendship is mutual, the graph used here is undirected.

The Social Network Friend Finder is a small, self-contained model of this idea. It allows a user to add people to a network, connect them as friends, find the friends two people have in common, discover how closely two people are connected, and explore the whole network with graph traversal algorithms.

The core logic is written in C++ using the Standard Template Library (STL). To make the behaviour easy to see and demonstrate, a web-based visual interface was also developed. It draws the network as a graph and animates BFS and DFS step by step. Both implementations follow the same algorithms and produce the same results.

The project covers the following topics of Advance Data Structure: graph representation using an adjacency list, Breadth-First Search, Depth-First Search, the queue and recursion (stack) data structures, set operations, and time and space complexity analysis.

2. PROBLEM STATEMENT

To model users and the connections between them as a graph, and to build a program that can explore a small social network. The program must allow a user to add users, create connections, find mutual friends and explore connections, while demonstrating graph traversal and relationship modelling using BFS and DFS.

The main questions the program must answer are:

- How can users and friendships be stored so that friends can be looked up quickly and new connections added easily?
- Which friends do two users have in common?
- How many steps (degrees) apart are two users, and what is the shortest chain of friends between them?
- Which users are reachable from a given user, and in what order are they discovered?
- Which people might a user know, based on the friends of their friends?

3. OBJECTIVES

- To represent a social network as an undirected graph using an adjacency list.
- To implement operations for adding users and creating friendships with proper validation (duplicates, unknown users, self-friendship).
- To find mutual friends of two users using set intersection.
- To implement Breadth-First Search to find the shortest connection path between two users and to traverse the network level by level.
- To implement Depth-First Search to explore the network in depth.
- To suggest new friends using the friends-of-friends approach.
- To analyse the time and space complexity of every operation.
- To build a visual web interface that demonstrates the algorithms step by step.

4. DATA STRUCTURE SELECTION AND JUSTIFICATION

Graph representation. A graph can be stored as an adjacency matrix or as an adjacency list. A social network is *sparse*: each person knows only a small fraction of all users. An adjacency matrix would reserve V×V cells even when almost all are empty, whereas an adjacency list stores only the connections that actually exist.

Criteria	Adjacency matrix	Adjacency list (chosen)
Space	$O(V^2)$	O(V + E)
List all friends of a user	O(V)	O(d), where d is the number of friends
Check if two users are friends	O(1)	O(log d) with a set
Add a new user	Resize the whole matrix	Insert one entry
Suited to	Dense graphs	Sparse graphs such as social networks

Data structures used in the program:

- **map<string, set<string>>** – the adjacency list. The map associates each user name with the set of that user's friends. A set avoids duplicate friendships automatically and keeps friends sorted, which makes the output of every traversal predictable and allows set_intersection to be used for mutual friends.
- **queue** – used by BFS. Its first-in, first-out order makes the search visit users in order of distance from the starting user, which is why the first path found to a target is a shortest path.
- **Recursion (call stack)** – used by DFS. Each recursive call moves one step deeper into a chain of friends, and returning from a call backtracks.
- **vector<string>** – stores a path (a list of users) in the BFS queue and stores the visit order for printing.
- **set<string> visited** – records users that have already been discovered, so that no user is processed twice and cycles in the network do not cause infinite loops.

5. ALGORITHM AND FLOWCHART

5.1 Algorithm

- **Add user (name):** if name already exists, report an error; otherwise create an empty friend set for it.
- **Add connection (a, b):** reject if a or b does not exist, if a equals b, or if they are already friends; otherwise insert b into friends(a) and a into friends(b), because friendship is mutual.
- **Mutual friends (a, b):** compute the intersection of friends(a) and friends(b) and print it.
- **Shortest path, BFS (source, target):** put source in visited and the path [source] in a queue. While the queue is not empty, remove the front path and let node be its last user. If node is the target, print the path and its degrees (length minus one) and stop. Otherwise, for each unvisited friend of node, mark it visited and enqueue the path extended with that friend. If the queue empties, no connection exists.
- **Suggest friends (user):** for each friend f of the user and each friend g of f, add g to the suggestions if g is not the user and not already a friend.
- **BFS traversal (start):** mark start visited and enqueue it; repeatedly dequeue a user, record it, and enqueue its unvisited friends.
- **DFS traversal (start):** DFS(node) marks node visited, records it, and calls DFS(friend) for every unvisited friend; call DFS(start).

5.2 Flowchart

Figure 1 shows the overall flow of the program and Figure 2 the flow of the BFS shortest-path search.

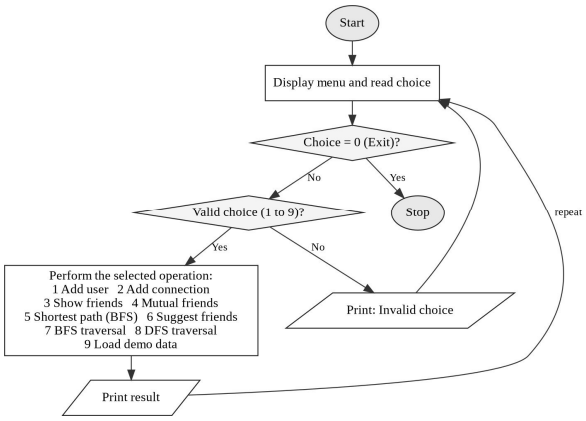


Figure 1: Overall flowchart

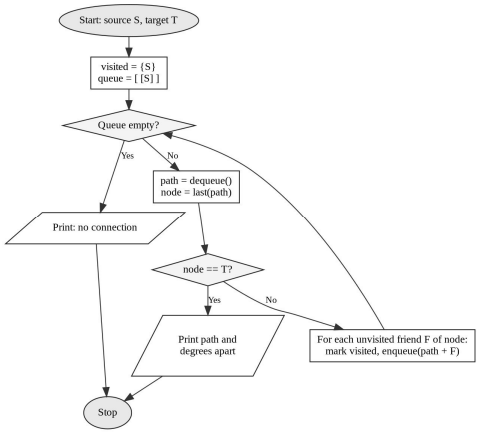


Figure 2: BFS shortest-path flowchart

6. WORKING IMPLEMENTATION (C++ SOURCE CODE)

The complete, working source code (friend_finder.cpp) is given below. It compiles with g++ - std=c++17 and runs as a menu-driven console program.

```
// Social Network Friend Finder
// Concepts: Graph (adjacency list), BFS, DFS

#include<algorithm> #include
<iostream> #include <map>
#include <queue> #include
<set> #include <string>
#include <vector> using
namespace std;

class SocialNetwork {
        map<string, set<string>> graph;  // user -> friends (sorted) bool exists(const string& u) const {

return graph.count(u) > 0; }

        void dfs(const string& node, set<string>& visited, vector<string>& order) { visited.insert(node);
order.push_back(node);
                for (const string& nxt : graph[node])
                        if (!visited.count(nxt)) dfs(nxt, visited, order);
}

        static void printList(const vector<string>& v, const string& sep) {
for (size_t i = 0; i < v.size(); i++) cout << (i ? sep : "") << v[i]; cout << "\n";
}

public:                 void addUser(const
string& name) {
                if (exists(name)) { cout << "'" << name << "' already exists.\n"; return; } graph[name] = {};
                cout << "User '" << name << "' added.\n";
}

        void addConnection(const string& a, const string& b) {
                if (!exists(a) || !exists(b)) { cout << "Both users must exist first.\n"; return; } if (a == b) { cout << "A user cannot befriend
                themselves.\n"; return; }
                if (graph[a].count(b)) { cout << a << " and " << b << " are already friends.\n"; return; } graph[a].insert(b);
                graph[b].insert(a);   // undirected: friendship is
mutual                          cout << a << " and " << b << " are now friends.\n";
                                }

        void showFriends(const string& name) {
                if (!exists(name)) { cout << "User not found.\n"; return; } cout << name << "'s friends: ";
                if (graph[name].empty()) { cout << "none\n"; return; }
```

```
        printList(vector<string>(graph[name].begin(), graph[name].end()), ", ");                }

    void mutualFriends(const string& a, const string& b) {
        if (!exists(a) || !exists(b)) { cout << "Both users must exist.\n"; return; } vector<string> common;
        set_intersection(graph[a].begin(),  graph[a].end(),
                                graph[b].begin(), graph[b].end(), back_inserter(common)); cout << "Mutual friends of "
        << a << " and " << b << ": ";
        if (common.empty()) cout << "none\n"; else
printList(common, ", ");
    }
    // BFS: shortest connection path
    void bfsPath(const string& start, const string& target) {
        if (!exists(start) || !exists(target)) { cout << "Both users must exist.\n"; return; } set<string> visited = {start};
       queue<vector<string>> q;
            q.push({start});
while (!q.empty()) {
            vector<string> path = q.front(); q.pop();
            const string node = path.back(); if (node ==
            target) {
                cout << "Shortest connection: ";
                for (size_t i = 0; i < path.size(); i++) cout << (i ? " -> " : "") << path[i]; cout << "    (" << path.size() - 1 << " degree(s)
                apart)\n";
                return;
            }
            for (const string& nxt : graph[node]) { if
(!visited.count(nxt)) {
visited.insert(nxt); vector<string> np = path;
np.push_back(nxt);
                    q.push(np);
                }
            }
        }
        cout << "No connection between " << start << " and " << target << ".\n";                }

    // Friends-of-friends who aren't already friends void suggestFriends(const
string& name) {
        if (!exists(name)) { cout << "User not found.\n"; return; } set<string> suggestions;   for
(const string& f : graph[name]) for (const string&
fof : graph[f])
            if (fof != name && !graph[name].count(fof)) suggestions.insert(fof); cout << "Suggestions for " <<
        name << ": ";
if (suggestions.empty()) cout << "none\n";
        else printList(vector<string>(suggestions.begin(), suggestions.end()), ", ");              }

    void bfsTraversal(const string& start) {
        if (!exists(start)) { cout << "User not found.\n"; return; } set<string> visited = {start};
        queue<string> q; q.push(start);
vector<string> order;                           while
(!q.empty()) {                          string node
= q.front();
        q.pop();
order.push_back(node);
        for (const string& nxt : graph[node])
            if (!visited.count(nxt)) { visited.insert(nxt); q.push(nxt); }
        }
        cout << "BFS from " << start << ": "; printList(order, " -> ");
}

    void dfsTraversal(const string& start) {
        if (!exists(start)) { cout << "User not found.\n"; return; } set<string> visited;
vector<string> order;                           dfs(start, visited,
order);                         cout << "DFS from "
<< start << ": ";
        printList(order, " -> ");
}

    void loadDemo() {
        for (string u : {"Alice", "Bob", "Charlie", "David", "Eve", "Frank"}) graph[u];
vector<pair<string, string>> edges = {{"Alice", "Bob"}, {"Alice", "Charlie"},
{"Bob", "David"}, {"Charlie", "David"},
                                                                {"David", "Eve"}};
        for (auto& e : edges) { graph[e.first].insert(e.second); graph[e.second].insert(e.first); } cout << "Demo network loaded: Alice, Bob, Charlie,
        David, Eve, Frank.\n";
    }
```

```
};

static string readLine(const string& prompt) { cout << prompt;
string s; getline(cin,s); return
s; }

int main() {
        SocialNetwork net; while (true) {
                cout << "\n===== Social Network Friend Finder =====\n"
                        "1. Add user\n2. Add connection\n3. Show friends\n4. Mutual friends\n" "5. Shortest connection path
                        (BFS)\n6. Suggest friends\n"
                        "7. BFS traversal\n8. DFS traversal\n9. Load demo data\n0. Exit\n"; string choice = readLine("Choose:
                ");
if (!cin) break;

                if (choice == "1") net.addUser(readLine("Name: "));
                else if (choice == "2") { string a = readLine("User 1: "); net.addConnection(a, readLine("User 2: ")); } else if (choice == "3") net.showFriends(readLine("User:
                "));
                else if (choice == "4") { string a = readLine("User 1: "); net.mutualFriends(a, readLine("User 2: ")); } else if (choice == "5") { string a = readLine("From: ");
                net.bfsPath(a, readLine("To: ")); }
                else if (choice == "6") net.suggestFriends(readLine("User: ")); else if (choice == "7")
net.bfsTraversal(readLine("Start user: ")); else if (choice == "8") net.dfsTraversal(readLine("Start user:
"));
                else if (choice == "9") net.loadDemo();
                else if (choice == "0") { cout << "Goodbye!\n"; break; } else cout << "Invalid
                choice.\n";
        }
return 0; }
```

7. SAMPLE INPUT AND OUTPUT

The demo network (menu option 9) contains six users and five friendships. Frank has no friends, so he is disconnected from the rest of the network.

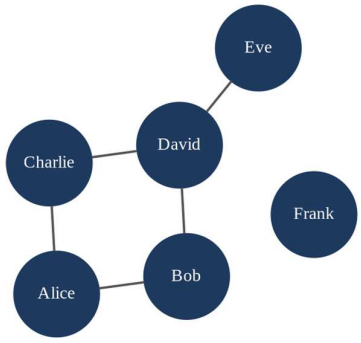


Figure 3: Demo network (Alice, Bob, Charlie, David, Eve, Frank)

7.1 Operations on the demo network

Menu option and input	Output
3 – Show friends: Alice	Alice's friends: Bob, Charlie
4 – Mutual friends: Bob, Charlie	Mutual friends of Bob and Charlie: Alice, David
5 – Shortest path: Alice to Eve	Shortest connection: Alice -> Bob -> David -> Eve (3 degree(s) apart)
5 – Shortest path: Alice to Frank	No connection between Alice and Frank.
6 – Suggest friends: Alice	Suggestions for Alice: David
7 – BFS traversal: Alice	BFS from Alice: Alice -> Bob -> Charlie -> David -> Eve
8 – DFS traversal: Alice	DFS from Alice: Alice -> Bob -> David -> Charlie -> Eve

The two traversals visit the same users in a different order. BFS visits Charlie before David because both of Alice's friends are explored before going further out, whereas DFS follows the chain Alice, Bob, David before returning to Charlie.

7.2 A custom session (user input and validation)

```
Choose: 1       Name: Riya          -> User 'Riya' added.                (Kabir, Meera added the same way) Choose: 1
Name: Riya      -> 'Riya' already exists.
Choose: 2       Riya, Kabir         -> Riya and Kabir are now friends.                (Kabir, Meera likewise) Choose: 2
                Riya, Riya          -> A user cannot befriend themselves.
Choose: 2       Riya, Kabir         -> Riya and Kabir are already friends. Choose: 3
                Kabir               -> Kabir's friends: Meera, Riya
Choose: 5       Riya, Meera         -> Shortest connection: Riya -> Kabir -> Meera
                                       (2 degree(s) apart)
Choose: 6       Riya                -> Suggestions for Riya: Meera Choose: 0
                -> Goodbye!
```

The session shows that duplicate users, self-friendship and repeated friendships are rejected, and that Meera is suggested to Riya because they share the friend Kabir.

8. TIME COMPLEXITY ANALYSIS

Let V be the number of users, E the number of connections and d the number of friends of a user. Because the map and the sets are ordered (balanced trees), a single lookup or insertion costs a logarithmic factor.

Operation	Technique	Time complexity	Space
Add user	Insert into map	O(log V)	O(1)
Add connection	Two set insertions	O(log V + log d)	O(1)
Show friends	Iterate over a set	O(log V + d)	O(d)
Mutual friends	set_intersection	$O(d_1 + d_2)$	$O(\min(d_1, d_2))$
Shortest path	BFS with queue	O((V + E) log V) *	O(V + E)
Suggest friends	Friends of friends	$O(d^2 \log V)$	O(V)
BFS traversal	Queue	O((V + E) log V) *	O(V)
DFS traversal	Recursion	O((V + E) log V) *	O(V)

* Each of the V users is processed once and each of the E edges is examined twice (once from each end), giving O(V + E) steps. The extra log V factor appears because the visited set and the map are ordered containers. With hash-based containers the traversals run in the classical O(V + E).

Space complexity. The adjacency list stores V entries and 2E friend references, so the graph needs O(V + E) space. BFS needs O(V) extra space for the queue and the visited set. DFS needs O(V) for the visited set and for the recursion stack in the worst case, for example a long chain of friends.

9. PROJECT REPORT AND DEMONSTRATION

To demonstrate the algorithms visually, a web application was built as a single HTML page with JavaScript. It implements the same data structure and algorithms as the C++ program and needs no installation: it opens in any modern browser.

9.1 Features of the web interface

- **Live graph:** users are nodes and friendships are lines; the layout re-arranges as the network changes, and clicking a node lists that user's friends.

- **Build and Explore tabs:** add users, connect users, load the demo network; find mutual friends, the shortest path (highlighted), friend suggestions, and animated BFS or DFS from any user.
- **Adjacency list panel and explanations:** shows the data structure updating live, with a "How it works" section and the complexity table.

9.2 Screenshots

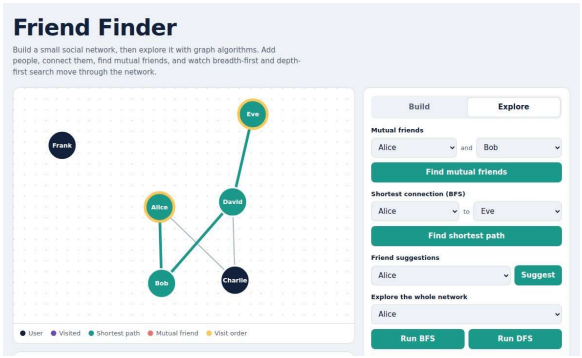


Figure 4: Shortest path Alice to Eve (BFS)

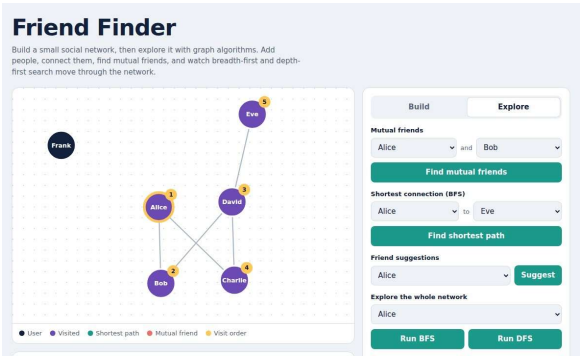


Figure 5: DFS from Alice, with visit order

9.3 How to run

C++ program: compile with g++ -std=c++17 -o friend_finder friend_finder.cppand run ./friend_finder. Choose option 9 to load the demo data, then try options 3 to 8.

Web interface: open friend-finder.htmlin a web browser. **9.4 Test cases**

No.	Test case	Expected result	Result
1	Add a user that already exists	Error message, no change	Pass
2	Connect a user with themselves, or twice	Rejected with a message	Pass
3	Mutual friends of Bob and Charlie	Alice, David	Pass
4	Shortest path Alice to Eve; Alice to Frank	3 degrees; no connection	Pass
5	BFS and DFS from Alice; suggestions for Alice	Different orders, same users; David	Pass

10. CONCLUSION

The Social Network Friend Finder successfully models a small social network as an undirected graph using an adjacency list. It supports adding users, creating connections, finding mutual friends, suggesting friends and exploring the network with BFS and DFS. The tests confirm that BFS finds shortest connection paths and that DFS reaches the same users in a different, depth-first order.

The project shows why the choice of data structure matters: an adjacency list keeps the memory proportional to the number of real friendships, a queue gives BFS its level-by-level behaviour, and recursion gives DFS its depth-first behaviour. Together with the visual web interface, it gave us a practical understanding of graph traversal, relationship modelling and complexity analysis.

Future scope: removing users and friendships, saving and loading the network from a file, finding connected components and communities, ranking suggestions by the number of mutual friends, and using weighted edges to represent the strength of a relationship.

11. REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022 (chapters on Elementary Graph Algorithms: BFS and DFS).

[2] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed., Pearson, 2013.

[3] E. Horowitz, S. Sahni and D. Mehta, *Fundamentals of Data Structures in C++*, 2nd ed., Universities Press, 2008.

[4] cppreference.com, C++ Standard Library reference: std::map, std::set, std::queue andstd::set_intersection. https://en.cppreference.com

[5] GeeksforGeeks, Breadth First Search and Depth First Search for a Graph. https://www.geeksforgeeks.org

[6] Course material for Advance Data Structure (25CSH-202), Chandigarh University.